Smriti Reddy Uravakonda

Boston, MA | (857) 746-3569 | u.smritireddy@gmail.com | LinkedIn | GitHub

EDUCATION

Northeastern University | M.S. Computer Science, GPA 3.6/4.0 | Boston, MA — May 2026
Dayananda Sagar University | B.Tech Computer Science, GPA 3.7/4.0 | Bangalore, India — June 2024

EXPERIENCE

ITDR, Inc. | Trainee Associate, Software Engineering | Remote, USA — Aug 2026 - Present
- Built a modular Python CLI application with JSON persistence, custom input validation, and 13 unit tests.
- Wrote regression tests and root-cause documentation for debugging exercises; completed SDLC and Git onboarding.

Northeastern University | Independent Study, CourtAccess AI | Boston, USA — Jan 2026 - Apr 2026
Advisor: Prof. Ramin Mohammadi | *Team repo:* Link
- Built Firebase authentication and role-based access control (public, court official, interpreter, admin) for the FastAPI backend of a team-built platform for real-time speech and document translation for the Massachusetts Trial Court.
- Wrote unit tests for the OCR, PII-detection, and speech modules of a codebase gated by GitHub Actions CI.

National Institute of Advanced Studies | Research Apprentice (Part-time) | Bangalore, India — Jul 2023 - Jun 2024
- Built a Python ML pipeline over 15GB of satellite imagery; SVM and Random Forest reached ~90% land-use accuracy.
- Parallelized preprocessing across 8 CPU cores (~25% faster); published at IEEE ASIANCON 2024.

Mekhalyn Consulting | Software Engineering Intern | Bangalore, India — Jan 2024 - May 2024
- Built a Node.js and SQL Server REST backend for a recruitment platform: pooling and query optimization cut API response time ~20% and raised throughput ~35%; composite indexes on 500K+ row tables cut slow queries ~40%.
- Set up GitHub Actions CI/CD, cutting the feature delivery cycle from about three weeks to under two days.

PROJECTS

NexLedger, Distributed Transaction System Link — Apr 2026 - Jun 2026
- Built an event-sourced payment ledger in Java (Spring WebFlux, R2DBC) that sustained 537 req/s under Gatling load with 0 failures over 81K requests (p95 32 ms), with zero projection/replay mismatches across 39K events afterward.
- Secured writes under concurrency with per-account Redis locks, optimistic versioning, a transactional outbox, idempotent projections, and crash-safe transfer sagas, tested with Testcontainers on real Postgres, Redis, and Kafka.
- Deployed on Kubernetes (two replicas) with zero failed requests across rolling restarts under load.
- Diagnosed an outbox ordering bug that surfaced only with two replicas and silently dropped events from the read model (232.00 instead of 300.00 in the test), then fixed it with a Postgres advisory lock and a regression test.

Fault-Tolerant Distributed Key-Value Store (Raft) Link — Aug 2025 - Dec 2025
- Implemented Raft from scratch in C++ (pre-vote leader election, log replication, replicated state machine, exactly-once client sessions) on a 5-node cluster, with a deterministic core shared by a fault-injecting simulator and a TCP server.
- Verified linearizability with a Porcupine-style checker on 1.9M operations under kill -9 and network partitions plus 200 simulated seeds; failover took a median of 139 ms (100-200 ms timeouts) and tests run clean under ASan and UBSan.
- Profiled a lock-contention bottleneck (84% of lock-held time was fsync and network I/O) and moved I/O out of the lock, raising write throughput 3.4x at 32 clients (7.5x at 64) and also stopping load-induced leader changes.

Argus, Distributed Observability Platform Link — Feb 2026 - Jun 2026
- Built a monitoring platform on Kafka, Spark Structured Streaming, TimescaleDB, and Elasticsearch across 3 instrumented microservices, with 10 s p50/p95/p99 windows, a trace-based dependency graph, and a live React dashboard.
- Detected 12/12 injected faults end to end (median 9.9 s from fault to alert) using an Isolation Forest and a PyTorch LSTM autoencoder served via ONNX Runtime; models versioned in the MLflow registry with PSI drift monitoring.
- Diagnosed a Kafka producer blocking the event loop on every ack, cutting request latency from 5-7 s to ~0.1 s.
- Tuned Spark shuffle partitions from 200 to 4, cutting micro-batches from ~5 min to ~10 s.

PUBLICATIONS

Optimizing Computation Offloading for Mobile Edge Devices using Particle Swarm Optimisation, IEEE ICAECT 2024.
Environmental Impact Analysis using Satellite Image Processing, IEEE ASIANCON 2024.

TECHNICAL SKILLS

- **Programming Languages:** Java, Python, C++, JavaScript, TypeScript, SQL
- **Backend & Distributed Systems:** Spring Boot, Spring WebFlux, FastAPI, Node.js, REST APIs, microservices, Kafka, Raft
- **Databases:** SQL (PostgreSQL, SQL Server, TimescaleDB), NoSQL (Redis, Elasticsearch)
- **Cloud & DevOps:** AWS (EC2, S3, Lambda, ECS), Docker, Kubernetes, CI/CD (GitHub Actions), Git
- **Frameworks & Tools:** Testcontainers, Gatling, pytest, React, Spark Structured Streaming, MLflow
- **Security & Authentication:** Firebase Authentication, role-based access control, OTP verification, SHA-256 hashing
- **Fundamentals:** data structures & algorithms, object-oriented design, concurrency, CQRS, event sourcing, sagas
- **Certifications:** AWS Academy Cloud Architecting and Cloud Foundations, Google Cloud Compute Basics